



PROJECT OVERVIEW

Power Glove Vision Project Overview

Architecture, controls, security, deployment, and project status.

2026 EDITION / IAIN BENNETT / MIT LICENSE

PowerGlove Vision lets you play RetroPie games by moving your hand in front of a camera connected to an Arduino UNO Q. Use your bare hand or a plain glove; there are no glove electronics to build. The UNO Q tracks your movements and sends controller input to a Raspberry Pi, where RetroArch sees a virtual gamepad named **PowerGlove Vision**.

The project includes eleven profiles: nine reusable Programs A-I and dedicated controls for Bad Street Brawler and Super Glove Ball. RetroPie can select a profile automatically when you launch a registered game. Glove Academy mode lets you practise without sending input to the cabinet.

The cabinet currently uses [lr-fceumm](#) as its default NES emulator. [lr-nestopia](#) has not been tested with PowerGlove Vision. Native Power Glove support remains planned work. The software preserves analogue and finger measurements for that future integration, but the current gamepad path does not provide every original glove feature.

Choose a guide

You want to...	Read...
Install both devices and play your first game	Installation Guide
Find a command or connection reminder	Quick Reference
Learn a game's gestures and try a short challenge	Gameplay Guide
Choose or experiment with Programs A-I	Programs A-I manual
Understand components and data flows	Architecture
Change settings or look up command flags	Configuration Reference
Understand network and pairing boundaries	Security policy
Change the project or its documentation	Contributing guide
Check dependency provenance or release history	Third-party components and Changelog

Quick start

These steps describe the current [dev](#) version. You need a working RetroPie installation, an UNO Q, a UVC-compatible camera, a powered USB hub, and a development computer with Arduino App Lab. Both devices need the same trusted network and internet access during setup. Keep a physical controller available for RetroArch configuration. Replace the example hostnames with your own.

1. Download the source on your development computer

- 1 Open a macOS or Linux terminal in the folder where you keep projects.
- 2 Run `command -v git python3 bash rsync zip`. Install any missing tools before continuing.
- 3 Run the commands below. `git clone` downloads the repository from GitHub; `cd` opens the downloaded folder.

```
SH
git clone --branch dev https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
```

2. Build the UNO Q package

- 1 In the same terminal and project folder, run the commands below.
- 2 Confirm that verification reports **App Lab installation ZIP verified**.

```
SH
scripts/build-app-lab-package.sh
python3 scripts/verify-app-lab-package.py
```

The package is `output/app-lab/PowerGlove-Vision-Uno-Q.zip`. It is separate from the GitHub source download and is the file you import into App Lab.

3. Install on the UNO Q

- 1 Connect the UNO Q by USB, open App Lab, and complete board setup. Record the board's hostname and join the same network as RetroPie.
- 2 Import the ZIP from step 2, open **PowerGlove Vision**, and select **Run**. App Lab transfers the application and matrix sketch to the board.
- 3 Connect the camera to the UNO Q through the powered hub. Use App Lab over the network as needed.
- 4 From your computer, connect with `ssh arduino@UNO-Q-NAME.local`. In that remote terminal, run the commands below. If the directory does not exist, stop and check the imported app's location; setup requires this exact path.

```
SH
cd /home/arduino/ArduinoApps/powerglove-vision
sudo python3 scripts/setup-machine.py uno-q
```

The installer completes host setup, installs the shutdown helper, and sets the app to start at boot. Correct any **FAIL** result. Pairing and gameplay **ACTION** messages are expected. Run `exit` to leave the remote terminal.

4. Install on the Raspberry Pi

- 1 Open a terminal on the Raspberry Pi running RetroPie, using its keyboard and display or an existing SSH connection.
- 2 Run the commands below on that Raspberry Pi. This downloads a second checkout and installs the receiver; the copy on your development computer cannot receive game input for RetroPie.
- 3 Correct any **FAIL** result before pairing. If the checkout already exists, open it instead of cloning again.

```
SH
sudo apt update
sudo apt install -y git
cd ~
git clone --branch dev https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local
```

The installer copies the required files into `/opt/powerglove-src`, installs commands under `/opt/powerglove/bin`, and adds the receiver, timer, RetroArch mapping, and launch hooks. No separate file copy is needed. Existing settings and tokens are preserved.

5. Pair the devices

- 1 On RetroPie, run `sudo /opt/powerglove/bin/powerglove-pair` and leave it running.
- 2 In your computer's browser, open <https://UNO-Q-NAME.local:8443/setup>.
- 3 Enter the RetroPie hostname and the printed one-time code. Select **Prepare one-time code**.
- 4 Compare the matrix **ID** with the beginning of the browser certificate's SHA-256 fingerprint. If they match, select the confirmation checkbox, enter the six-digit matrix PIN, and complete pairing.
- 5 Confirm that the RetroPie command reports completion. If the code expires, restart the pairing command and prepare another attempt.

6. Check the camera and game controls

- 1 Open <http://UNO-Q-NAME.local:8088/learn>. Wait for the camera view and practise the gestures. The first activation can take several minutes to prepare the runtime.
- 2 Return to Dashboard, select a profile, and use **Calibrate** if your resting hand position causes unwanted movement. Select **Start controller** when ready.
- 3 In RetroArch, use your physical controller to open **Settings > Input > RetroPad Binds > Port 1 Controls**. Select **PowerGlove Vision**, check the bindings, and save any changes.
- 4 Launch a registered game. Confirm its profile on Dashboard and test movement, A, B, Start, and Select. End the game and confirm that gestures turn off.
- 5 Check that the app returns after a normal reboot and that the RetroPie receiver timer is enabled. The receiver starts 45 seconds after boot; controller transmission still requires **Start controller**.

The [Installation Guide](#) provides checkpoints, recovery steps, alternative pairing, and the exact startup checks. The [command reference](#) explains every project flag and the system-command options used in these steps.

Controls

Calibration records the resting hand position that the app treats as the centre of movement. Move away from that position to give a direction and return to it to release that direction. Recalibrate after moving the camera or changing your playing position. Some profiles replace ordinary hand movement with wrist steering or other controls, as shown below.

Across the profiles, hold a **V sign** for about 0.7 seconds to send Start and a **thumbs-up with the other fingers closed** to send Select. These poses suppress A/B attacks; some profiles can still generate directional or auxiliary input, so keep your hand near its resting position while using them.

Programs A-I

These reusable mappings produce ordinary NES controls. You do not need to launch Bad Street Brawler first. The table describes controller output; its effect depends on the game. A pulsed button repeatedly presses and releases.

Program	Movement	Actions and special gestures
A - Pinball	Ordinary movement is disabled.	Index curl sends A; thumb curl sends Up; wrist roll sends B. Pulling back toggles combined flippers.
B - Joust	Move your hand left or right.	Index or middle curl pulses A; thumb curl holds B.
C - Gyryss	Roll your wrist left or right.	A straight index finger holds A; pulling back sends B. Use the game's Attack Control B mode.
D - Challenge	All four hand-movement directions are reversed.	Thumb curl sends A; index curl sends B.
E - Defender II	Move your hand in four directions.	Thumb curl sends A; wrist roll sends B; ring-finger curl rapidly alternates left and right.
F - Sesame Street	Ordinary directional output is disabled.	Moving an open hand away from centre sends A; closing all fingers sends B.
G - Gun Smoke	Move your hand in four directions; wrist roll adds left or right.	Index curl sends A; pushing forward sends B. Combine them for A+B. Thumb and ring-finger curl together suppress D-pad and A/B output.
H - General	Move your hand in four directions.	Index curl pulses A; thumb curl pulses B.
I - Knight Rider	Roll your wrist to steer; lower your hand to brake.	Index curl sends Up for acceleration; pushing sends Up+A for turbo; thumb curl sends B.

Programs A, D, and H have no default ROM assignment. Choose one on Dashboard to try it, then register the exact game filename if you want automatic selection. The [Programs manual](#) includes illustrations; the [Gameplay Guide](#) adds objectives and tips.

Bad Street Brawler

Gesture	Controller output
Move your hand left, right, up, or down	Corresponding D-pad direction
Curl your thumb	Pulsed B
Curl your middle finger	A+B
Roll your wrist left or right	A plus that direction
Push toward the camera	Glove Zap: short simultaneous Left + Right pulse

Glove Zap uses the cartridge's simultaneous Left + Right command. The Bad Street Brawler profile emits a 180 ms pulse per forward push; release before trying again. FCEUmm must allow opposing directions for this game only. See the [configuration reference](#). The extra `BTN_TR2` signal remains available, but needs no RetroArch assignment for this action.

Super Glove Ball

Gesture	Controller output
Move your hand left, right, up, or down	Corresponding D-pad direction
Curl your index finger	A
Curl your thumb	B

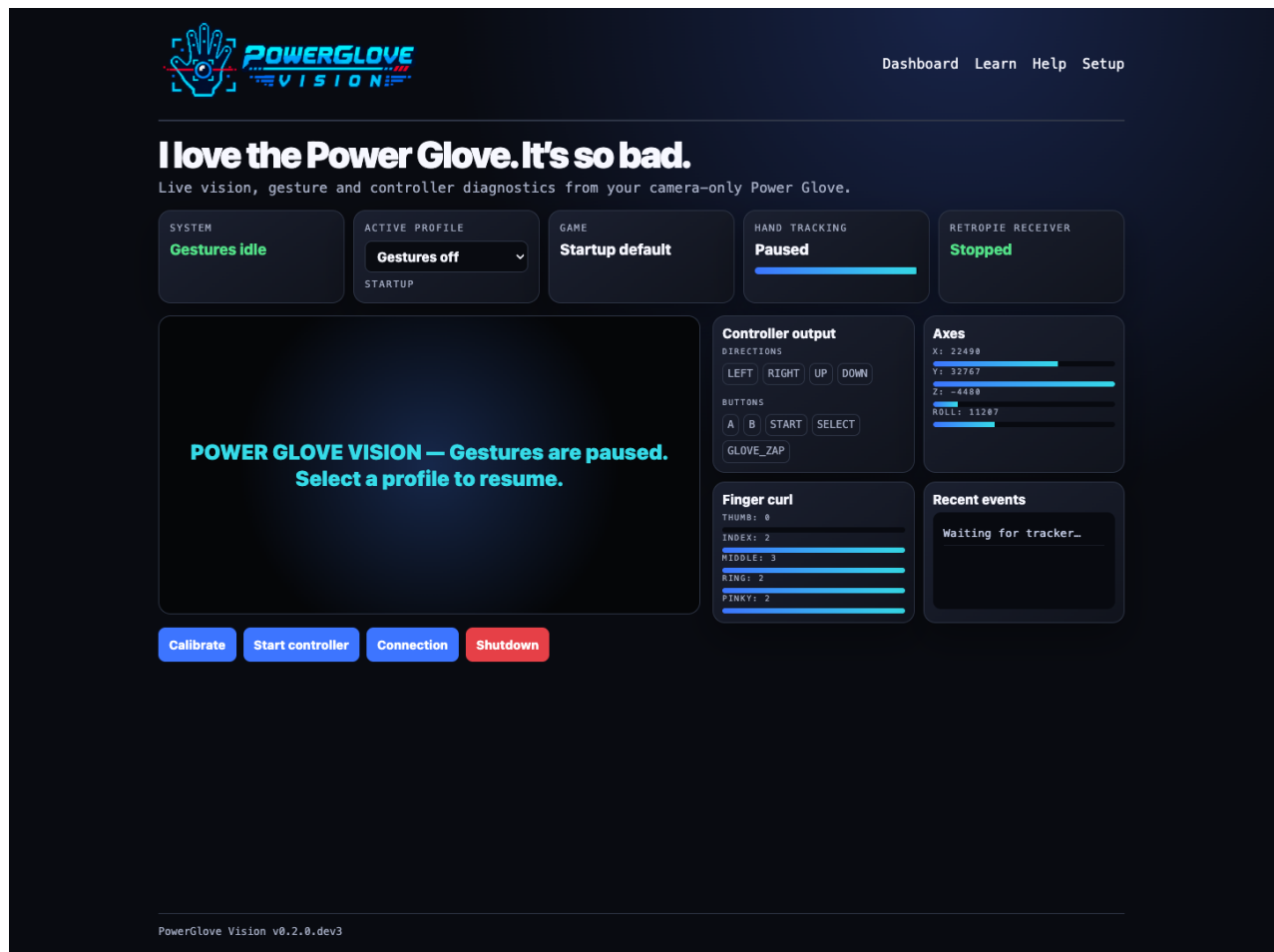
The protocol also carries hand position, estimated depth, wrist roll, and finger measurements for future native-glove support.

Use the web interface

Page	What it does
Dashboard, /dashboard	Shows the camera and generated inputs; selects the current profile and starts or stops delivery.
Glove Academy, /learn	Provides twelve practice lessons and guided gesture tuning, with game input paused.
Help, /help	Opens the local manuals and PDFs; This cabinet shows current connection details.
Setup, /setup	Saves connection, camera, and startup settings; the Games section edits RetroPie mappings with backup and restore. Pairing requires HTTPS on port 8443.

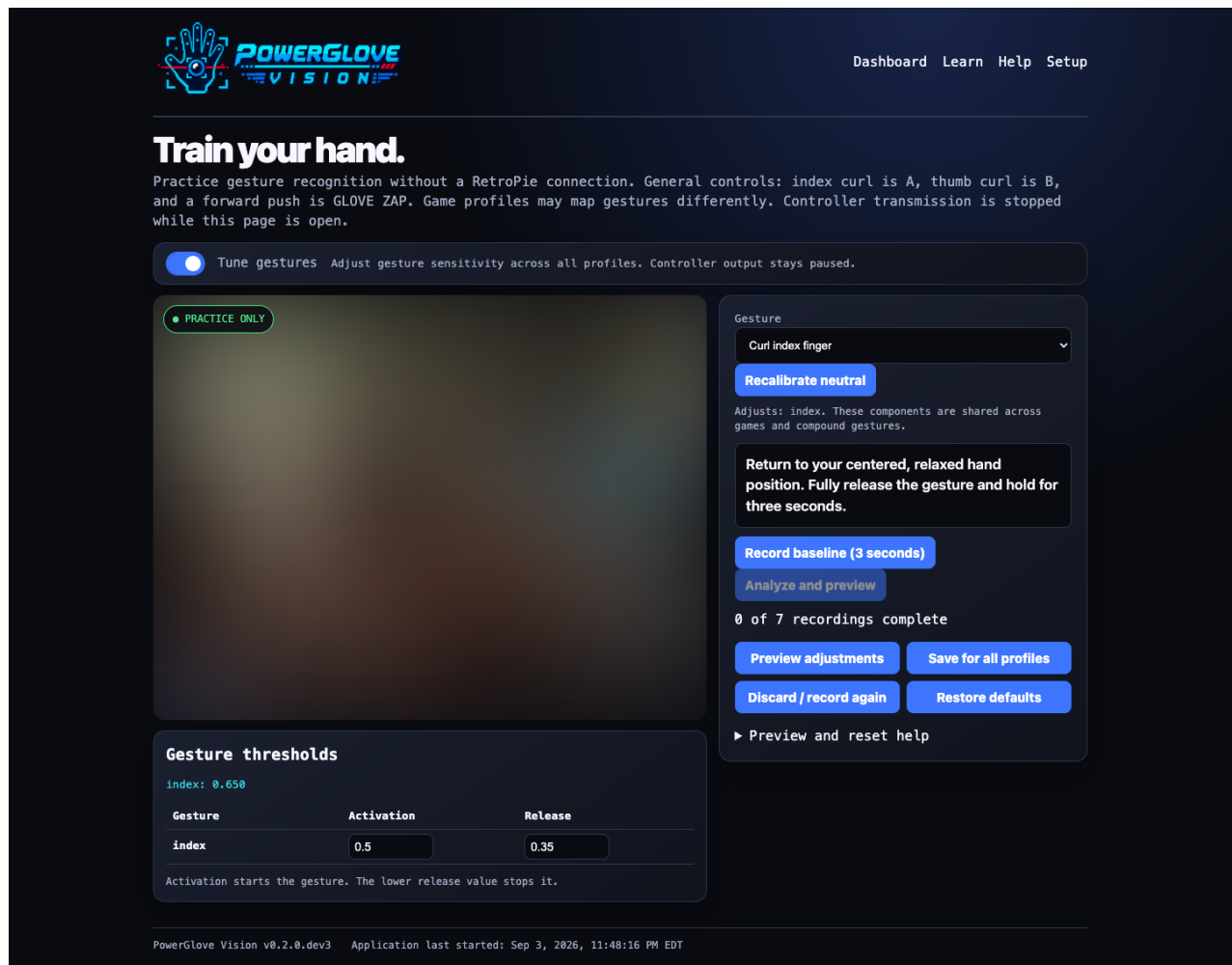
The application preloads OpenCV and MediaPipe in the background while the website is available. This prepares gesture tracking without opening the camera. With **Gestures off** selected, capture starts only when you choose an active profile or open Glove Academy. If preloading is still underway, activation waits for it to finish. See the [startup details](#) for measured results and troubleshooting.

Stop controller pauses delivery while leaving active tracking available. **Gestures off** closes the camera. **Shutdown** requests a Linux halt, but the tested UNO Q restarts afterward. A disappearing website is not proof that it is safe to remove power. See the installation guide before using Shutdown.



Dashboard showing the selected profile and controller readings

The screenshots below show the current interface. Camera imagery is blurred for privacy.



Glove Academy in Tune mode, with thresholds below the blurred camera

In **Glove Academy**, switch on **Tune gestures** to adjust sensitivity. The UNO Q shows a scanning **T** during tuning and a matching scanning **L** during ordinary practice. Both modes pause game input.

Tuning uses three recordings of three seconds each: open hand, gesture, open hand. Optional **Set up my hand** uses open hand, gentle fist with the thumb outside, open hand to measure all five fingers. Preview before saving. Saved Activation and Release thresholds apply in gameplay across profiles. **Glove Zap** and **Pull Back** have separate practice lessons and tuning controls; movement tuning ends by returning to the starting position and camera distance.

Games

Edit the game mappings installed on your RetroPie. Saving affects the next game launch.

Use the exact ROM filename, including its extension: `Joust (USA).7z` and `Joust (USA).nes` need separate entries. Matching ignores letter case. Only NES and Famicom launches use these mappings.

► Available profile identifiers

Game mappings JSON

```
{
  "games": {
    "Bad Street Brawler (USA).nes": "bad_street_brawler",
    "Bad Street Brawler (USA).zip": "bad_street_brawler",
    "Bad Street Brawler (USA).7z": "bad_street_brawler",
    "Super Glove Ball (USA).nes": "super_glove_ball",
    "Super Glove Ball (USA).zip": "super_glove_ball",
    "Super Glove Ball (USA).7z": "super_glove_ball",
    "Joust (USA).nes": "program_b",
    "Gyruss (USA).nes": "program_c",
    "Defender II (USA).nes": "program_e",
    "Sesame Street 1-2-3 (USA).nes": "program_f",
    "Gun.Smoke (USA).nes": "program_g",
    "Knight Rider (USA).nes": "program_i",
    "Joust (USA).zip": "program_b",
    "Joust (USA).7z": "program_b",
    "Gyruss (USA).zip": "program_c",
    "Gyruss (USA).7z": "program_c",
    "Defender II (USA).zip": "program_e",
    "Defender II (USA).7z": "program_e",
    "Sesame Street 1-2-3 (USA).zip": "program_f",
    "Sesame Street 1-2-3 (USA).7z": "program_f",
    "Gun.Smoke (USA).zip": "program_g"
```

Validate

Format

Save

Reload

Download backup

Restore previous save

Installed mappings loaded.

Games editor in the lower part of Setup

Scroll down **Setup** to **Games** to map exact ROM filenames to profiles. Saving affects the next game launch, not the game already running.

Maintain or extend the project

Use the [Installation Guide's maintenance section](#) for updates, and the [Configuration Reference](#) for settings and all command options. The [Contributing guide](#) covers tests, documentation, package verification, and releases. Printable editions are stored in [output/pdf/](#).

PowerGlove Vision is an independent project licensed under the [MIT License](#). Nintendo, NES, Power Glove, and the named games belong to their respective owners. Third-party software and models retain their own terms, documented in [Third-party components](#).